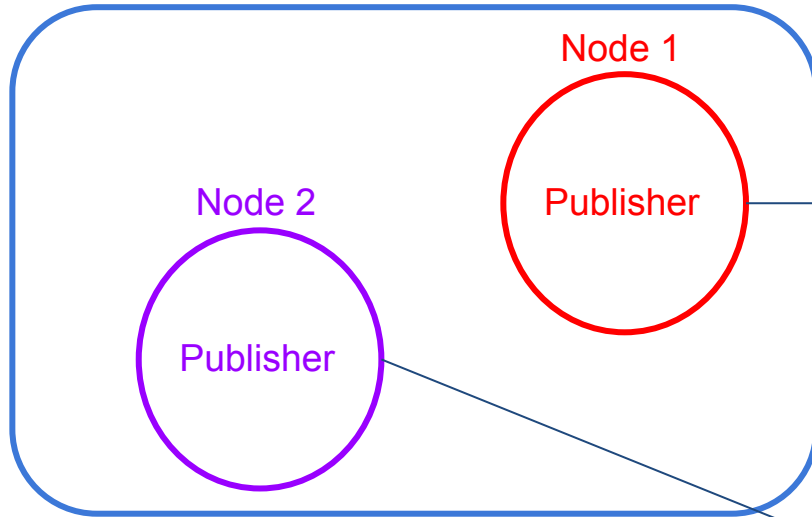


Previously on IA368 II

 ROS2

Packages, nodes, topics, messages...

Package 1



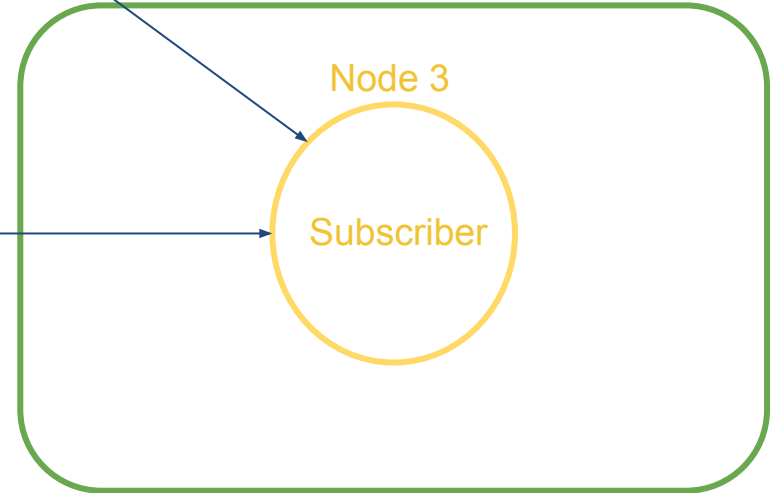
/topic1

Message 1

/topic2

Message 2

Package 2



Let's work!

Let's create our own workspace!

<https://docs.ros.org/en/jazzy/Tutorials/Beginner-Client-Libraries/Creating-Your-First-ROS2-Package.html>

<https://docs.ros.org/en/jazzy/Tutorials/Beginner-Client-Libraries/Writing-A-Simple-Py-Publisher-And-Subscriber.html>

Time was
up
So today

Let's work!

Let's create our own workspace!

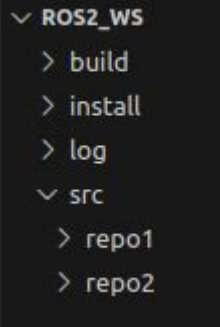
<https://docs.ros.org/en/jazzy/Tutorials/Beginner-Client-Libraries/Creating-Your-First-ROS2-Package.html>

<https://docs.ros.org/en/jazzy/Tutorials/Beginner-Client-Libraries/Writing-A-Simple-Py-Publisher-And-Subscriber.html>

But faster!

Workspace - what is a workspace ?

- A workspace is a directory containing ROS 2 packages
- By default, ROS 2 commands are not on your `PATH` and cannot be found
- When you “source” the file `/opt/ros/jazzy/setup.bash`, you put `ros2` commands on the `PATH` so that you can use them
- We can create a workspace with our own packages (including packages from Github)
- By convention, the workspace looks like:

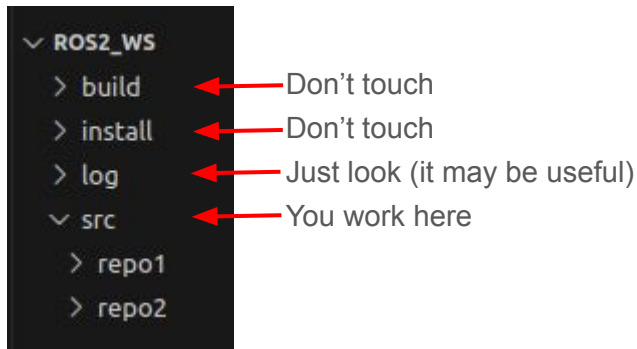


```
▼ ROS2_WS
  > build
  > install
  > log
  ▼ src
    > repo1
    > repo2
```

Workspace - what is a workspace ?

- Build artifacts are stored in `ros2_ws/build`
- The final results are installed to `ros2_ws/install`, which is where your code actually runs from
- Log files generated from building the packages go in `ros2_ws/log`. The latest logs are in `ros2_ws/log/latest`
- The source code to a ROS 2 packages are placed under `ros2_ws/src/<repo>`

- Basically:



Workspace - how to create a workspace ?

- Best practice is to create a new directory for every new workspace
- The name doesn't matter, but it is helpful to have it indicate the purpose of the workspace
- In one terminal, create the directory:

```
mkdir -p ros2_ws/src
```

```
cd ros2_ws/src
```

Let's work!

Let's create a package!

(just the publisher)

<https://docs.ros.org/en/jazzy/Tutorials/Beginner-Client-Libraries/Writing-A-Simple-Py-Publisher-And-Subscriber.html>

CoppeliaSim + ROS 2



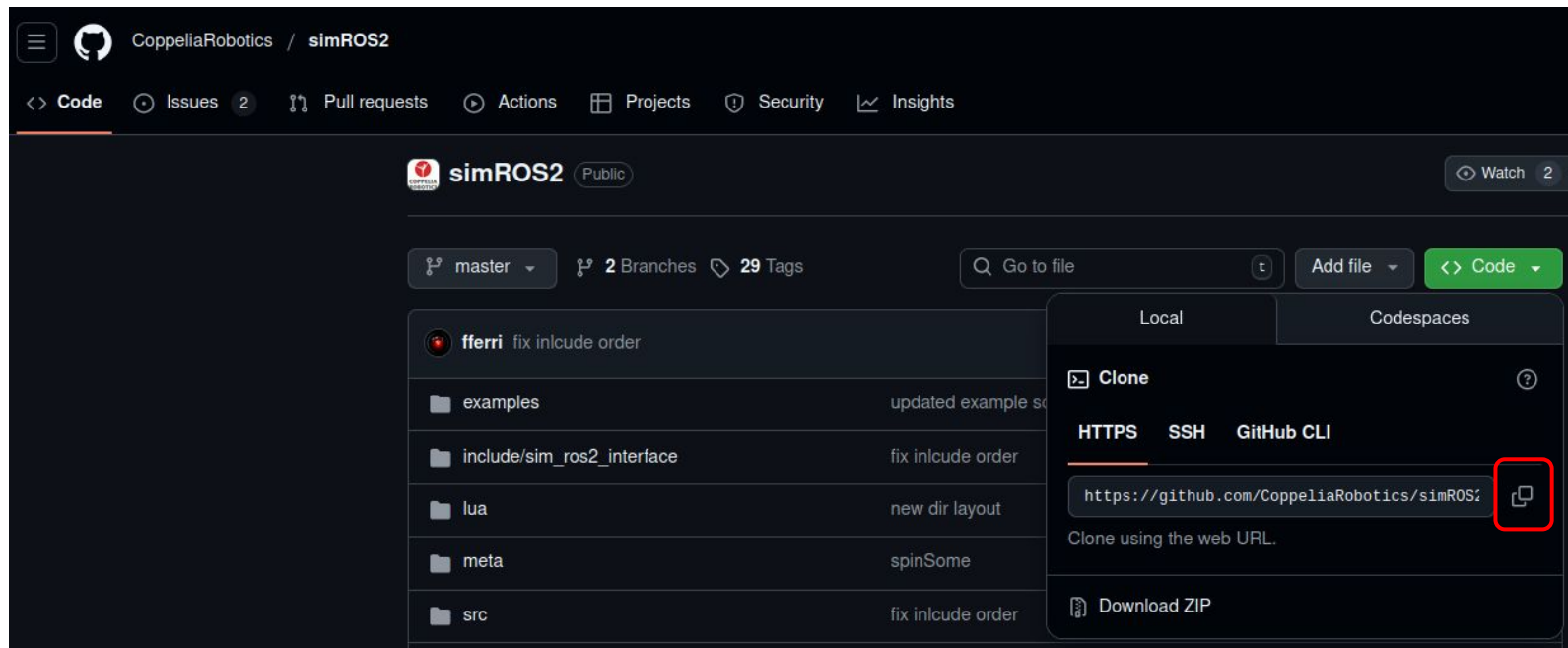
 ROS2

CoppeliaSim + ROS 2

- ROS 2 functionality in CoppeliaSim is supported via the [ROS 2 Interface](#) (libsimROS2.so), and is part of the CoppeliaSim API framework
- Inside a CoppeliaSim script (Python or Lua), the ROS 2 plugin is loaded with `simROS2 = require('simROS2')`
- The plugin is open source and can be modified as much as needed in order to support a specific feature
- For example, if you need a specific message, you could add it and recompile it by yourself
- It will be compiled to the "libsimROS2.so" file, and that is used by CoppeliaSim

Let's build !

<https://github.com/CoppeliaRobotics/simROS2>



CoppeliaRobotics / simROS2

<> Code Issues 2 Pull requests Actions Projects Security Insights

simROS2 Public Watch 2

master 2 Branches 29 Tags

Go to file Add file <> Code

ffferri fix include order

- examples updated example s
- include/sim_ros2_interface fix include order
- lua new dir layout
- meta spinSome
- src fix include order

Local Codespaces

Clone ?

HTTPS SSH GitHub CLI

https://github.com/CoppeliaRobotics/simROS2

Clone using the web URL.

Download ZIP

CoppeliaSim + ROS 2

- Inside the src directory in your ROS 2 workspace, clone the repository:

```
cd ros2_ws/src
```

```
git clone https://github.com/CoppeliaRobotics/simROS2.git  
sim_ros2_interface
```

```
cd sim_ros2_interface
```

```
git checkout coppeliasim-v4.10.0-rev0
```

- Replace coppeliasim-v4.10.0-rev0 with the actual CoppeliaSim version you have
- Let's edit `meta/interfaces.txt` to include messages

CoppeliaSim + ROS 2

- Using your favorite text editor, include `sensor_msgs/msg/CameraInfo` and `sensor_msgs/msg/RegionOfInterest` to the list
- Inside your ROS 2 workspace (before `src`), provide your CoppeliaSim directory as an environment variable and build the package:

```
cd ../../..
```

```
export
```

```
COPPELIASIM_ROOT_DIR=/home/patric_moreto/Unicamp/Pós/2º2025/CoppeliaSim_Edu_V4_10_0_rev0_Ubuntu22_04
```

```
colcon build --symlink-install --packages-select  
sim_ros2_interface
```

CoppeliaSim + ROS 2

- After every build process, don't forget to source your updated workspace:
`source install/setup.bash`
- We just compiled the "libsimROS2.so" file again, but with a new message included
- Next time you open CoppeliaSim, you're able to use this [message struct](#) with a publisher/subscriber mechanism
- Let's learn how to create a publisher, a subscriber and how to fill a message using the `ros2InterfaceTopicPublisherAndSubscriber-python.ttt` scene

CoppeliaSim + ROS 2

- Using the `ros2InterfaceTopicPublisherAndSubscriber-python.ttt` scene, if you check in Rviz, you'll see that the images are upside down
- To solve it, you can edit the sensor script, adding the line `data = sim.transformImage(data,resolution,4)` in line 16
- Now, the images from passiveSensor are upside down. To solve it, you can edit the sensor script again, adding the line `data = sim.transformImage(msg.data.tobytes(), [msg.width, msg.height], 4)` in line 36
- Also, edit line 37 to `sim.setVisionSensorImg(self.passiveVisionSensor, data)`

CoppeliaSim + ROS 2 + python/MATLAB/C++

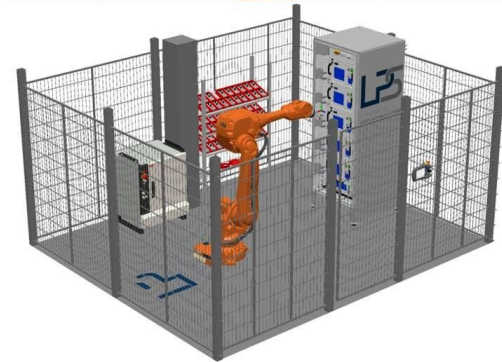
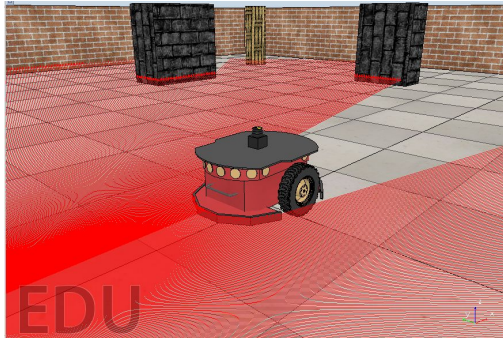


 ROS2

The ROS 2 logo, consisting of a 3x3 grid of dots to the left of the text "ROS2" in a dark blue, sans-serif font.

CoppeliaSim + ROS 2 + python/MATLAB/C++

- Use in simulation then real life



Real robot+ ROS 2 + python/MATLAB/C++



ROS2



CoppeliaSim + ROS 2 + python/MATLAB/C++

- With all the data being read from the simulator just need to act in the simulation
- Legacy API (Remote API) or ZeroMQ
 - It enable to communicate with the simulator through python
 - For example, the speed for each wheel:
 - “sim.setJointTargetVelocity(jointHandle, vel)”

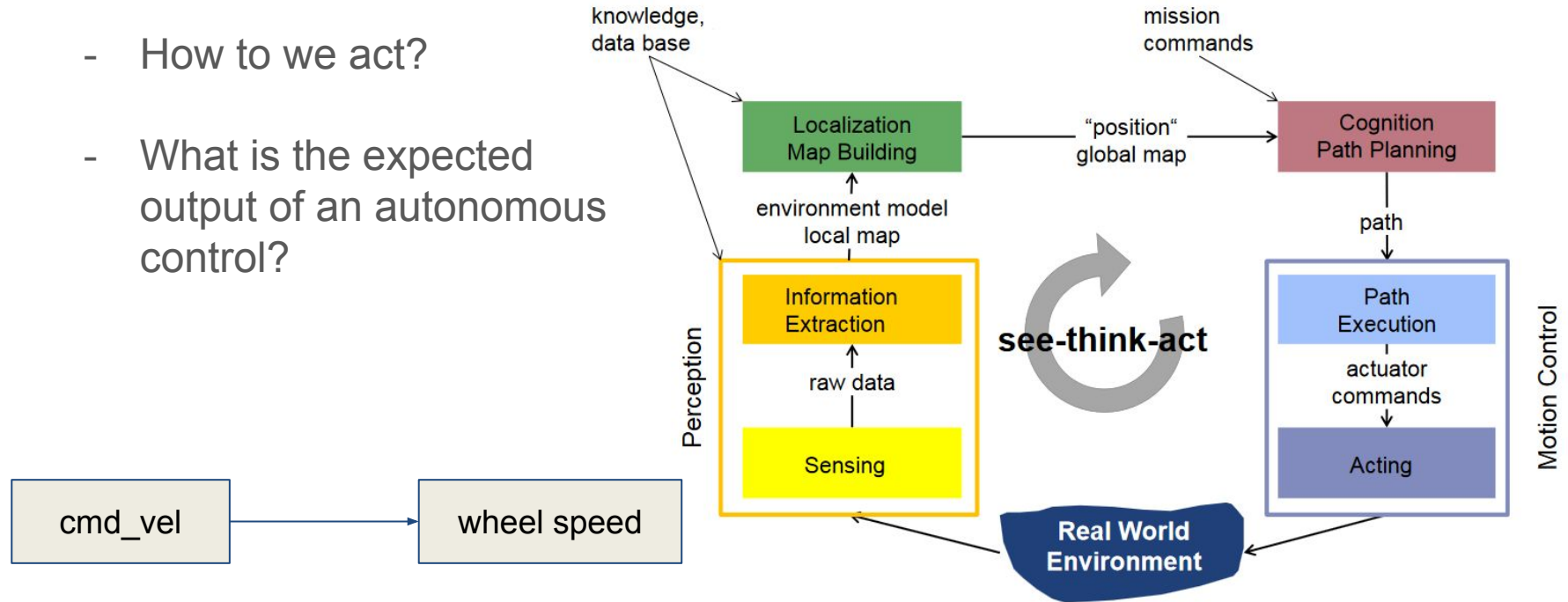
```
pip install coppeliasim-zmqremoteapi-client
```

ZeroMQ: <https://github.com/CoppeliaRobotics/zmqRemoteApi>



CoppeliaSim + ROS 2 + python/MATLAB/C++

- How to we act?
- What is the expected output of an autonomous control?



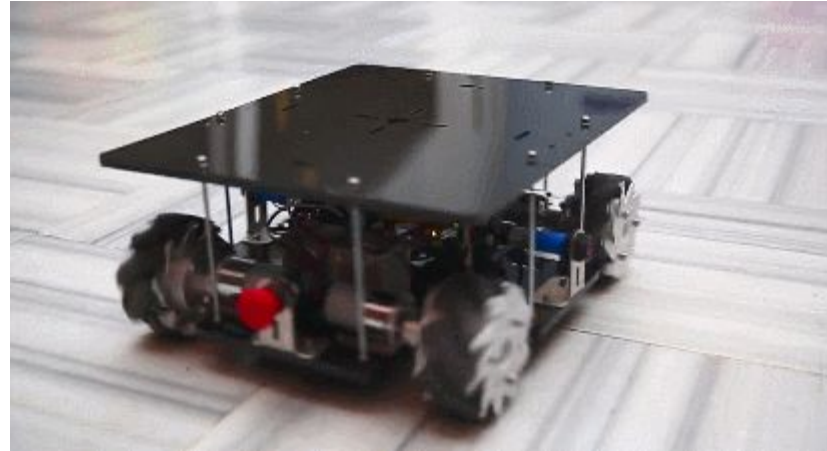
Let's Work!

```
from coppeliasim_zmqremoteapi_client import  
RemoteAPIClient
```

```
client = RemoteAPIClient()  
sim = client.require('sim')  
sim.loadScene(SCENE_PATH)  
sim.startSimulation()
```

```
·  
·  
·  
·
```

```
sim.stopSimulation()
```



Control your robot through ROS

Thank You...

...for your attention

